

Browser Tycoon MVP: Spec-Driven Implementation Document

1. Product Goal

Build a browser-based, fully local, single-player strategy MVP inspired by RollerCoaster Tycoon.

The MVP is a dashboard strategy game, not a construction simulator. The player chooses a market, builds rides, sets ticket price, manages cash and debt, and advances the simulation month by month.

The game should create one core tension:

Can the player build a profitable amusement park by matching ride choices to market demand while managing cash, land, pricing, and debt?

2. Platform Requirements

- Runs fully in browser.
 - No backend.
 - No login.
 - No external database.
 - Game state should persist in localStorage.
 - Player should be able to start a new game and resume an existing game.
 - Use simple UI components.
 - Prioritize transparent simulation feedback over visual polish.
-

3. Core Game Loop

One turn equals one month.

Loop:

1. Player selects location.
2. Player starts with fixed cash, land, and no rides.
3. Player builds rides.
4. Player sets ticket price.
5. Player optionally takes or repays loans.
6. Player runs next month.
7. Game calculates:
 8. visitors
 9. revenue

10. maintenance cost
11. loan interest
12. net profit
13. satisfaction
14. reputation
15. park value
16. Player reads monthly report.
17. Player adjusts strategy.
18. Repeat.

Core success loop:

Better ride mix → higher attractiveness → more visitors → more revenue → more cash → bigger rides → stronger park.

Core failure loop:

Wrong ride mix → low satisfaction → weak reputation → fewer visitors → low revenue → maintenance pressure → debt spiral.

4. MVP Scope

Include

- Location selection.
- Two starting locations.
- Hidden market variables.
- Visible market ratings.
- Ride catalogue.
- Land capacity.
- Ride construction.
- Monthly simulation.
- Ticket pricing.
- Cash.
- Loans.
- Maintenance costs.
- Satisfaction.
- Reputation.
- Park value.
- Monthly report.
- Save/load using localStorage.

Exclude

- Actual coaster building.
- Pathfinding.
- Staff hiring.
- Shops as buildable entities.
- Guest-level simulation.
- Weather.

- Terrain.
 - Ride breakdowns.
 - Visual park map.
 - Animations.
 - Competitor actions.
 - Multiplayer.
-

5. Game Start

Player sees two location cards.

Location A: Oakville

Visible to player:

- Competition: 1 / 5
- Visitor Spending: 2 / 5
- Tourism: 1 / 5

Description:

A growing town with few entertainment options. Visitors are easy to attract, but budgets are limited.

Hidden values:

```
{
  "id": "oakville",
  "name": "Oakville",
  "population": 220000,
  "touristsPerMonth": 8000,
  "competitionStrength": 0.15,
  "spendingPower": 0.85,
  "growthRateMonthly": 0.012,
  "familyShare": 0.65,
  "thrillShare": 0.25,
  "touristShare": 0.10
}
```

Location B: Coast City

Visible to player:

- Competition: 4 / 5
- Visitor Spending: 4 / 5
- Tourism: 5 / 5

Description:

A bustling tourist hotspot. Visitors spend freely, but rival parks compete for their attention.

Hidden values:

```
{
  "id": "coast_city",
  "name": "Coast City",
  "population": 520000,
  "touristsPerMonth": 130000,
  "competitionStrength": 0.55,
  "spendingPower": 1.35,
  "growthRateMonthly": 0.004,
  "familyShare": 0.35,
  "thrillShare": 0.40,
  "touristShare": 0.25
}
```

6. Initial Game State

At start:

```
{
  "month": 1,
  "cash": 500000,
  "debt": 0,
  "maxDebt": 1000000,
  "annualInterestRate": 0.08,
  "ticketPrice": 20,
  "landUsed": 0,
  "landCapacity": 30,
  "reputation": 0.5,
  "satisfaction": 0.7,
  "rides": [],
  "monthlyHistory": []
}
```

7. Ride Catalogue

Each ride is a business asset.

Attributes:

- id
- name
- category

- buildCost
- footprint
- monthlyMaintenance
- monthlyCapacity
- familyAppeal
- thrillAppeal
- touristAppeal
- excitement
- baseAttraction

MVP Ride List

```
[
  {
    "id": "carousel",
    "name": "Carousel",
    "category": "family",
    "buildCost": 80000,
    "footprint": 3,
    "monthlyMaintenance": 3000,
    "monthlyCapacity": 9000,
    "familyAppeal": 5,
    "thrillAppeal": 1,
    "touristAppeal": 2,
    "excitement": 2,
    "baseAttraction": 12
  },
  {
    "id": "mini_train",
    "name": "Mini Train",
    "category": "family",
    "buildCost": 120000,
    "footprint": 5,
    "monthlyMaintenance": 5000,
    "monthlyCapacity": 12000,
    "familyAppeal": 5,
    "thrillAppeal": 1,
    "touristAppeal": 3,
    "excitement": 2,
    "baseAttraction": 16
  },
  {
    "id": "ferris_wheel",
    "name": "Ferris Wheel",
    "category": "family",
    "buildCost": 220000,
    "footprint": 6,
    "monthlyMaintenance": 8000,
    "monthlyCapacity": 15000,
    "familyAppeal": 4,
    "thrillAppeal": 2,
```

```

    "touristAppeal": 4,
    "excitement": 3,
    "baseAttraction": 24
  },
  {
    "id": "swing_ride",
    "name": "Swing Ride",
    "category": "balanced",
    "buildCost": 180000,
    "footprint": 4,
    "monthlyMaintenance": 7000,
    "monthlyCapacity": 13000,
    "familyAppeal": 3,
    "thrillAppeal": 3,
    "touristAppeal": 3,
    "excitement": 4,
    "baseAttraction": 22
  },
  {
    "id": "drop_tower",
    "name": "Drop Tower",
    "category": "thrill",
    "buildCost": 320000,
    "footprint": 4,
    "monthlyMaintenance": 12000,
    "monthlyCapacity": 11000,
    "familyAppeal": 1,
    "thrillAppeal": 5,
    "touristAppeal": 3,
    "excitement": 5,
    "baseAttraction": 30
  },
  {
    "id": "roller_coaster",
    "name": "Roller Coaster",
    "category": "thrill",
    "buildCost": 650000,
    "footprint": 9,
    "monthlyMaintenance": 24000,
    "monthlyCapacity": 22000,
    "familyAppeal": 2,
    "thrillAppeal": 5,
    "touristAppeal": 5,
    "excitement": 5,
    "baseAttraction": 55
  },
  {
    "id": "water_ride",
    "name": "Water Ride",
    "category": "premium",
    "buildCost": 500000,

```

```

    "footprint": 8,
    "monthlyMaintenance": 20000,
    "monthlyCapacity": 18000,
    "familyAppeal": 4,
    "thrillAppeal": 3,
    "touristAppeal": 5,
    "excitement": 4,
    "baseAttraction": 45
  },
  {
    "id": "mega_coaster",
    "name": "Mega Coaster",
    "category": "premium",
    "buildCost": 1200000,
    "footprint": 12,
    "monthlyMaintenance": 45000,
    "monthlyCapacity": 35000,
    "familyAppeal": 1,
    "thrillAppeal": 5,
    "touristAppeal": 5,
    "excitement": 5,
    "baseAttraction": 90
  }
]

```

8. Player Actions

8.1 Build Ride

Rules:

- Player can build a ride if $\text{cash} \geq \text{buildCost}$.
- Player can build a ride if $\text{landUsed} + \text{footprint} \leq \text{landCapacity}$.
- Building a ride immediately subtracts buildCost from cash.
- Building a ride immediately increases landUsed.
- Multiple copies of the same ride are allowed.
- Duplicate rides should receive diminishing attraction value.

Error states:

- Insufficient cash.
- Insufficient land.

8.2 Set Ticket Price

Allowed values:

[10, 20, 30, 40, 50]

Rules:

- Player can change ticket price any time before running next month.
- Ticket price affects visitor demand and satisfaction.

8.3 Take Loan

Rules:

- Player can borrow in increments of 50000.
- Total debt cannot exceed maxDebt.
- Borrowed amount immediately increases cash.
- Borrowed amount immediately increases debt.

8.4 Repay Loan

Rules:

- Player can repay in increments of 50000.
- Player cannot repay more than current cash.
- Player cannot repay more than current debt.
- Repayment immediately reduces cash and debt.

8.5 Run Next Month

Rules:

- Simulation calculates the month.
- Cash updates by net profit.
- Reputation updates.
- Market population and tourists update by growth rate.
- Monthly report is appended to history.
- Month increments by 1.

9. Simulation Formulas

9.1 Market Demand

```
localDemand = population * 0.015;  
touristDemand = touristsPerMonth * 0.12;  
rawMarketDemand = localDemand + touristDemand;
```

9.2 Park Attraction

Each ride contributes attraction adjusted for market fit.


```
marketFit =  
    (ride.familyAppeal * location.familyShare) +  
    (ride.thrillAppeal * location.thrillShare) +  
    (ride.touristAppeal * location.touristShare);
```

```
rideAttraction = ride.baseAttraction * (marketFit / 5);
```

Apply duplicate penalty by ride type:

```
duplicateMultiplier = Math.max(0.4, 1 - duplicateIndex * 0.2);
```

Examples:

- First copy: 1.0
- Second copy: 0.8
- Third copy: 0.6
- Fourth copy and beyond: 0.4

```
totalAttraction = sum(rideAttraction * duplicateMultiplier);
```

9.3 Reputation Modifier

```
reputationModifier = 0.5 + reputation;
```

If reputation is 0.5, modifier is 1.0.

Reputation range:

```
0.1 to 2.0
```

9.4 Competition Modifier

```
competitionModifier = 1 - location.competitionStrength;
```

9.5 Ticket Price Modifier

Baseline ticket price is 20.

```
priceModifier = Math.max(0.35, 1 - ((ticketPrice - 20) * 0.025));
```

Examples:

- \$10 → 1.25
- \$20 → 1.00
- \$30 → 0.75
- \$40 → 0.50
- \$50 → 0.35

9.6 Visitor Demand Captured

```
attractionModifier = Math.min(1.5, totalAttraction / 100);
```

```
visitorsBeforeCapacity =  
    rawMarketDemand *  
    attractionModifier *  
    reputationModifier *  
    competitionModifier *  
    priceModifier;
```

9.7 Ride Capacity

```
totalRideCapacity = sum(ride.monthlyCapacity);
```

Final visitors:

```
visitors = Math.floor(Math.min(visitorsBeforeCapacity, totalRideCapacity));
```

If no rides exist:

```
visitors = 0;
```

9.8 Revenue

Ticket revenue:

```
ticketRevenue = visitors * ticketPrice;
```

In-park spend:

```
baseSpend = 8;  
averageSpend = baseSpend * location.spendingPower * satisfaction;  
inParkRevenue = visitors * averageSpend;
```

Total revenue:

```
totalRevenue = ticketRevenue + inParkRevenue;
```

9.9 Costs

Maintenance:

```
maintenanceCost = sum(ride.monthlyMaintenance);
```

Monthly loan interest:

```
monthlyInterest = debt * (annualInterestRate / 12);
```

Total cost:

```
totalCosts = maintenanceCost + monthlyInterest;
```

Net profit:

```
netProfit = totalRevenue - totalCosts;
```

Cash update:

```
cash = cash + netProfit;
```

9.10 Satisfaction

Satisfaction should measure perceived value, crowding, and ride-market fit.

Value score:

```
parkValueForGuests = totalAttraction / ticketPrice;  
valueScore = clamp(parkValueForGuests / 5, 0.2, 1.2);
```

Crowding score:

```
capacityUtilization = visitors / totalRideCapacity;  
crowdingScore = clamp(1.2 - capacityUtilization, 0.4, 1.0);
```

Market fit score:

```
averageMarketFit = average(marketFit / 5 across rides);  
marketFitScore = clamp(averageMarketFit, 0.4, 1.0);
```

Final satisfaction:

```
satisfaction = clamp(  
    (valueScore * 0.45) +  
    (crowdingScore * 0.25) +  
    (marketFitScore * 0.30),  
    0.1,  
    1.0  
);
```

If no rides exist:

```
satisfaction = 0.2;
```

9.11 Reputation

Reputation changes slowly based on satisfaction.

```
reputationChange = (satisfaction - 0.7) * 0.15;  
reputation = clamp(reputation + reputationChange, 0.1, 2.0);
```

Interpretation:

- Satisfaction above 70% improves reputation.
- Satisfaction below 70% damages reputation.

9.12 Market Growth

Each month:

```
population = population * (1 + growthRateMonthly);  
touristsPerMonth = touristsPerMonth * (1 + growthRateMonthly);
```

9.13 Park Value

```
rideAssetValue = sum(ride.buildCost * 0.7);  
profitValue = Math.max(0, netProfit * 24);  
parkValue = cash + rideAssetValue + profitValue - debt;
```

10. Monthly Report

After running a month, show:

- Month number
- Visitors
- Ticket revenue
- In-park revenue
- Total revenue
- Maintenance cost
- Loan interest
- Net profit
- Cash
- Debt
- Satisfaction
- Reputation
- Park value

Also show 1 to 3 explanation messages.

Examples:

- Visitors increased because your new rides improved total park attraction.
 - Satisfaction fell because ticket price is high compared to park attraction.
 - Profit fell because maintenance costs increased after expansion.
 - Visitor count was capped by ride capacity.
 - Your park is struggling against strong competition in this market.
 - Reputation improved because satisfaction was above 70%.
-

11. UI Screens

11.1 Start Screen

Elements:

- Game title.
- New Game button.
- Resume Game button if localStorage save exists.

11.2 Location Selection Screen

Display two location cards.

Each card contains:

- Location name.
- Description.
- Competition rating.
- Visitor Spending rating.

- Tourism rating.
- Start Game button.

Do not show hidden variables.

11.3 Main Dashboard

Top stats:

- Month
- Cash
- Monthly profit
- Visitors
- Satisfaction
- Reputation
- Park value
- Debt
- Land used / land capacity

Main sections:

1. Ride Catalogue
2. Built Rides
3. Pricing
4. Loans
5. Monthly Report
6. Run Next Month button

11.4 Ride Catalogue

Each ride card shows:

- Name
- Category
- Build cost
- Footprint
- Monthly maintenance
- Capacity
- Family appeal
- Thrill appeal
- Tourist appeal
- Build button

Build button disabled if:

- insufficient cash
- insufficient land

11.5 Built Rides

Show list of built rides.

Each row:

- Ride name
- Category
- Footprint
- Maintenance
- Capacity

11.6 Pricing Panel

Show:

- Current ticket price.
- Buttons for \$10, \$20, \$30, \$40, \$50.

11.7 Loan Panel

Show:

- Current debt.
- Maximum debt.
- Annual interest rate.
- Borrow \$50,000 button.
- Repay \$50,000 button.

11.8 Monthly Report Panel

Show latest month report and explanation messages.

12. State Shape

Use one centralized game state object.

```
type Location = {  
  id: string;  
  name: string;  
  description: string;  
  visibleRatings: {  
    competition: number;  
    visitorSpending: number;  
    tourism: number;  
  };  
  population: number;  
  touristsPerMonth: number;  
  competitionStrength: number;  
  spendingPower: number;  
  growthRateMonthly: number;  
  familyShare: number;  
  thrillShare: number;  
}
```

```

    touristShare: number;
};

type RideDefinition = {
  id: string;
  name: string;
  category: "family" | "thrill" | "balanced" | "premium";
  buildCost: number;
  footprint: number;
  monthlyMaintenance: number;
  monthlyCapacity: number;
  familyAppeal: number;
  thrillAppeal: number;
  touristAppeal: number;
  excitement: number;
  baseAttraction: number;
};

type BuiltRide = RideDefinition & {
  instanceId: string;
  builtMonth: number;
};

type MonthlyReport = {
  month: number;
  visitors: number;
  ticketRevenue: number;
  inParkRevenue: number;
  totalRevenue: number;
  maintenanceCost: number;
  loanInterest: number;
  netProfit: number;
  cash: number;
  debt: number;
  satisfaction: number;
  reputation: number;
  parkValue: number;
  landUsed: number;
  landCapacity: number;
  messages: string[];
};

type GameState = {
  selectedLocation: Location;
  month: number;
  cash: number;
  debt: number;
  maxDebt: number;
  annualInterestRate: number;
  ticketPrice: number;
  landUsed: number;

```



```
landCapacity: number;  
reputation: number;  
satisfaction: number;  
rides: BuiltRide[];  
latestReport: MonthlyReport | null;  
monthlyHistory: MonthlyReport[];  
};
```

13. Required Functions

Implement simulation as pure functions.

createNewGame(locationId)

Input:

- locationId

Output:

- GameState

Behavior:

- Loads location data.
- Initializes game state.
- Saves to localStorage.

buildRide(gameState, rideId)

Input:

- GameState
- rideId

Output:

- updated GameState

Rules:

- Throw or return error if insufficient cash.
- Throw or return error if insufficient land.
- Create BuiltRide with unique instanceId.
- Deduct buildCost.
- Increase landUsed.
- Save state.

setTicketPrice(gameState, price)

Input:

- GameState
- price

Output:

- updated GameState

Rules:

- Price must be one of allowed values.

takeLoan(gameState, amount)

Rules:

- Amount must be 50000.
- Debt cannot exceed maxDebt.
- Cash increases.
- Debt increases.

repayLoan(gameState, amount)

Rules:

- Amount must be 50000.
- Cash must be enough.
- Debt must be enough.

simulateNextMonth(gameState)

Input:

- GameState

Output:

- updated GameState with latestReport

Rules:

- Calculate all formulas.
- Update cash.
- Update satisfaction.
- Update reputation.
- Grow market.
- Append monthly report.
- Increment month.
- Save state.

saveGame(gameState)

Save serialized state to localStorage.

loadGame()

Load game state from localStorage.

resetGame()

Clear localStorage save.

14. Utility Functions

Implement these utilities:

```
clamp(value: number, min: number, max: number): number
formatMoney(value: number): string
formatPercent(value: number): string
generateId(): string
```

15. Acceptance Criteria

Game Start

- User can start a new game.
- User can select Oakville or Coast City.
- Hidden location values are not displayed.
- Visible ratings are displayed.

Ride Building

- User can build rides when cash and land are sufficient.
- Cash decreases by build cost.
- Land used increases by footprint.
- User cannot build ride when cash is insufficient.
- User cannot build ride when land is insufficient.

Pricing

- User can set ticket price to one of five values.
- Selected price is clearly visible.
- Price affects visitors and satisfaction after simulation.

Loans

- User can borrow \$50,000 increments.

- User cannot exceed max debt.
- User can repay \$50,000 increments.
- User cannot repay more than available cash or current debt.
- Loan interest affects monthly costs.

Monthly Simulation

- Run Next Month produces a monthly report.
- Visitors, revenue, costs, profit, satisfaction, reputation, and park value update.
- Monthly history stores each report.
- Month increments by one.
- State persists after page refresh.

Strategy Differences

- Oakville should perform better with family-heavy, lower-price strategies.
- Coast City should perform better with premium/thrill-heavy strategies.
- Duplicate ride spam should be less effective because of diminishing returns.
- High ticket prices should reduce visitors.
- High ticket prices should reduce satisfaction when park attraction is weak.

16. Suggested File Structure

```
src/  
  App.tsx  
  data/  
    locations.ts  
    rides.ts  
  game/  
    types.ts  
    constants.ts  
    simulation.ts  
    actions.ts  
    storage.ts  
  components/  
    StartScreen.tsx  
    LocationSelect.tsx  
    Dashboard.tsx  
    StatBar.tsx  
    RideCatalog.tsx  
    BuiltRides.tsx  
    PricingPanel.tsx  
    LoanPanel.tsx  
    MonthlyReport.tsx  
  utils/  
    format.ts  
    math.ts
```

17. Implementation Notes for Codex

Build the MVP in small steps.

Step 1: Create data models, location data, ride data, and initial game state.

Step 2: Create pure simulation function and unit-test it manually from sample game states.

Step 3: Create actions for build ride, set price, take loan, repay loan, and simulate month.

Step 4: Build UI screens.

Step 5: Connect localStorage persistence.

Step 6: Add monthly report explanation messages.

Step 7: Tune numbers only after the full loop works.

Do not start with styling. First make the loop playable.

18. Balance Targets

After 12 simulated months:

Oakville

A player using family rides and ticket price 20 should usually be profitable.

A player using only premium thrill rides should struggle early because of cost pressure.

Coast City

A player using cheap family rides should grow slowly due to competition.

A player using premium rides and ticket price 30 or 40 should be able to outperform if debt is managed well.

General

- A player should not be able to build the biggest ride immediately without taking debt.
 - Maintenance should matter.
 - Debt should be useful but risky.
 - A park with no new investment should plateau.
 - Duplicate ride spam should underperform a balanced ride mix.
-

19. Win and Lose Conditions

For MVP, do not hard-stop the game early.

Show soft goals:

- Reach park value of \$5,000,000.
- Reach monthly profit of \$250,000.
- Survive 24 months.
- Keep satisfaction above 75%.

Soft failure indicators:

- Cash below 0.
- Debt near maximum.
- Satisfaction below 50%.
- Reputation below 0.5.

If cash is negative and debt is maxed, show warning:

Your park is insolvent. You can continue playing, but you cannot build new rides until cash becomes positive.

20. Design Principle

Every month should teach the player something.

The monthly report should explain why numbers changed. The player should never feel that the simulation is a black box.

The game becomes fun when the player says:

“I know why this happened, and I have a plan for next month.”